PRIORITY QUEUES AND FAIR SHARING

WITH SQS AND LAMBDA



Salvatore Caputi

AWS User Group Venice – 2026-09-24

WHO AM I?



WHO AM I?



Solutions Architect
@Pixartprinting

Kenjutsu practitioner
@Bushido Dojo Venezia

Bojutsu practitioner
@Bushido Dojo Venezia

Iaido practitioner
@Bushido Dojo Venezia

THE CONTEXT

- Service that manipulates pdf files running on **Batch**
- **~15k** execution per day
- Execution duration from **few seconds to 50 minutes**
- **Cap** to concurrent executions

THE NEED

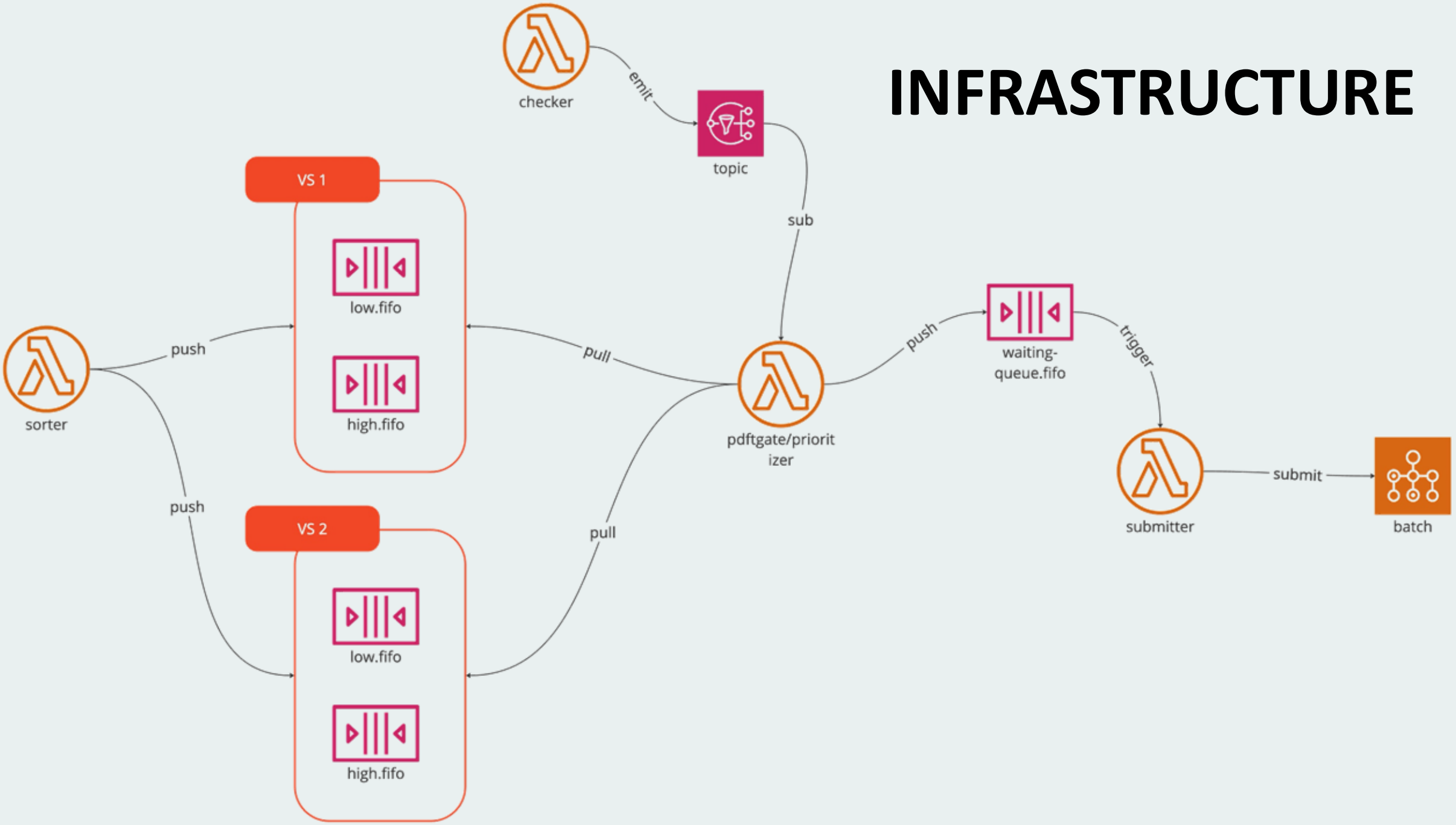
- Give precedence to **high priority** jobs (e.g. order that must be delivery today).

- Resources **equally shared** among several value streams.

WHY NOT BATCH?

- In 2022 we tested **priority jobs** and **fair sharing** features.
- It didn’t **work**.

INFRASTRUCTURE



SORTER

- Extract **tag** and **priority** from message
- Sort the message to proper **queue**
- If tag doesn’t match any queue, use a **default** queue

CHECKER

- Count the total of **pending jobs** in Batch queue
- Count the size of messages into **waiting** queue
- Sum them and compare with a given **threshold**
- If sum is lower, send an event to **Prioritizer**

PRIORITIZER

- Check if the message is **recent**
- Loop over all active **high-priority** queues until batchSize or maxAttempts has been reached
- If the batch has still room, loop over all active **low-priority** queues until batchSize or maxAttempts has been reached

PRIORITIZER

- Queues are **weighted**, so a value stream may have more resources than other

- A single loop, either high or low priority, reaches all active queues, this guarantee a **fair-sharing** among all value streams. This could result in a batch size greater than batchSize value.

PRIORITIZER

```
// GetNumberOfMessageToGet returns the maxNumberOfMessages to pass to the queue service.
// It calculates a fair weight for each queue based on the sum of all weight. To avoid starvation, 1 is forced
// as minimum in case of high unbalanced queues
func (p *Prioritizer) GetNumberOfMessageToGet(batchSize int64, weightSum int64, queueWeight int64) int64 {  3 us
    nMessages := (float64(batchSize) * float64(queueWeight)) / float64(weightSum)
    return int64(math.Max(math.Min(math.Round(nMessages), float64(MaxMessages)),  y: 1))
}
```

case	batch size	q	weight	weightSum	available in queue	requested	fetched	notes
one 3/2	3	q1	1	1	2	3	2	single queue
one 3/3	3	q1	1	1	3	3	3	single queue
two 4/4	4	q1	1	2	2	2	2	two equal queues equal availability
two 4/4	4	q2	1	2	2	2	2	two equal queues equal availability
two unbalanced	8	q1	1	2	10	4	4	q1 has plenty of messages
two unbalanced	8	q2	1	2	2	4	2	q2 requests 4 but only 2 available -> fetched capped by availability
two different weight	12	q1	2	3	10	8	8	double weight -> double share
two different weight	12	q2	1	3	10	4	4	

case	batch size	q	weight	weightSum	available in queue	requested	fetched	notes
many queues small batch - overshoot	3	q1	1	5	10	1	1	anti-starvation floor: 5 equal queues x 1 = 5 total > batchSize 3
many queues small batch - overshoot	3	q2	1	5	10	1	1	anti-starvation floor: 5 equal queues x 1 = 5 total > batchSize 3
many queues small batch - overshoot	3	q3	1	5	10	1	1	anti-starvation floor: 5 equal queues x 1 = 5 total > batchSize 3
many queues small batch - overshoot	3	q4	1	5	10	1	1	anti-starvation floor: 5 equal queues x 1 = 5 total > batchSize 3
many queues small batch - overshoot	3	q5	1	5	10	1	1	anti-starvation floor: 5 equal queues x 1 = 5 total > batchSize 3
two high unbalanced	10	q1	200	201	10	10	10	dominant queue capped at MaxMessages (10)
two high unbalanced	10	q2	1	201	10	1	1	anti-starvation floor: near-zero share still gets 1 -> total 11 > batchSize 10

WHAT IF…

A VS has corrupted messages	Disable polling from VS queues
A VS has huge number of messages	Increase weight / Disable other queues
Batch max concurrency has been increased	Increase batchSize
Batch needs to be stopped	Stop the prioritizer, messages are still enqueued
A request has been cancelled	Yet to be implemented ☺


T. HANKS